

BÁO CÁO BÀI TẬP: KỸ NĂNG VIẾT PROMPT HIỆU QUẢ

Họ và tên sinh viên: Ngô Phương Đông

Mã sinh viên: 25022825

Môn học: Nhập môn công nghệ số và ứng dụng AI

I. Mục tiêu

Phát triển và làm chủ kỹ năng "Prompt Engineering" (Kỹ thuật thiết kế câu lệnh) để tận dụng tối đa sức mạnh của các Mô hình ngôn ngữ lớn (LLMs) như ChatGPT, Gemini. Qua đó, áp dụng AI như một trợ lý học tập đắc lực trong việc tra cứu, tổng hợp kiến thức và ôn thi các môn chuyên ngành.

II. Yêu cầu & Thực hiện

Dưới đây là 3 tác vụ học tập phổ biến trong khối ngành Công nghệ thông tin cùng 3 phiên bản prompt từ cơ bản đến nâng cao cho mỗi tác vụ.

Tác vụ 1: Tóm tắt một tài liệu học thuật

(Chủ đề: Sự khác biệt giữa Machine Learning và Deep Learning)

(Tài liệu: <https://quantrimang.com/lang-cong-nghe/su-khac-biet-giua-ai-hoc-may-va-hoc-sau-157948>)

- Prompt cơ bản (Đơn giản, ngắn gọn): Tóm tắt bài viết sau: [Chèn tài liệu]

Tóm tắt bài viết sau: <https://quantrimang.com/lang-cong-nghe/su-khac-biet-giua-ai-hoc-may-va-hoc-sau-157948>

Dưới đây là tóm tắt ngắn gọn nội dung bài viết từ Quantrimang:

- **Tổng quan**
 - AI (Trí tuệ nhân tạo) là lĩnh vực lớn trong khoa học máy tính, mục tiêu tạo ra máy móc có khả năng "thông minh" như con người.
 - Machine Learning (Học máy) là một nhánh của AI, giúp máy học từ dữ liệu để đưa ra dự đoán/quyết định.
 - Deep Learning (Học sâu) là một kỹ thuật nâng cao của Machine Learning.
- 🔗 Quan hệ:
AI > Machine Learning > Deep Learning
- 📖 **Machine Learning (Học máy)**
 - Cho phép máy học từ dữ liệu và cải thiện theo thời gian mà không cần lập trình chi tiết.
 - Dựa vào thuật toán để:
 - Phân tích dữ liệu
 - Tìm quy luật
 - Dự đoán kết quả
 - Thường cần:
 - Dữ liệu (có thể nhỏ hơn Deep Learning)
 - Sự can thiệp của con người (chọn đặc trưng, giám sát...)
- 📖 **Deep Learning (Học sâu)**
 - Là một dạng Machine Learning dùng mạng nơ-ron nhiều lớp.
 - Tự động học đặc trưng từ dữ liệu (ít cần con người can thiệp).
 - Hoạt động tốt với:
 - Dữ liệu lớn
 - Dữ liệu phi cấu trúc (ảnh, âm thanh, video)
- 🔍 **Điểm khác biệt chính giữa Machine Learning và Deep Learning**

Tiêu chí	Machine Learning	Deep Learning
Bản chất	Nhánh của AI	Nhánh con của ML
Cách học	Cần con người chọn đặc trưng	Tự động học đặc trưng
Dữ liệu	Ít hơn, thường có cấu trúc	Rất lớn, đa dạng
Độ phức tạp	Thấp hơn	Cao hơn
Hiệu quả	Tốt với bài toán đơn giản	Tốt với bài toán phức tạp (ảnh, giọng nói...)
- ✅ **Kết luận**
 - AI là "bức tranh lớn".
 - Machine Learning là cách để AI học từ dữ liệu.
 - Deep Learning là phương pháp mạnh mẽ hơn, giúp máy học sâu và xử lý các bài toán phức tạp.

- **Prompt cải tiến (Chi tiết hơn, có cấu trúc):** Hãy tóm tắt đoạn văn bản sau để chỉ ra sự khác biệt giữa Machine Learning và Deep Learning. Yêu cầu tóm tắt ngắn gọn thành 4-5 gạch đầu dòng, tập trung vào khái niệm và yêu cầu về dữ liệu. Văn bản: [Chèn tài liệu]

Hãy tóm tắt đoạn văn bản sau. Sau đó chỉ ra sự khác biệt giữa Machine Learning và Deep Learning với 4-5 gạch đầu dòng, tập trung vào khái niệm và yêu cầu về dữ liệu. Văn bản: <https://quantrimang.com/lang-cong-nghe/su-khac-biet...>

Dựa trên nội dung từ bài viết bạn cung cấp và các tài liệu chuyên môn liên quan, dưới đây là tóm tắt văn bản và sự khác biệt giữa Machine Learning (ML) và Deep Learning (DL):

1. Tóm tắt đoạn văn bản

Văn bản giải thích mối quan hệ giữa trí tuệ nhân tạo (AI), Machine Learning (Học máy) và Deep Learning (Học sâu). Trong đó, AI là khái niệm rộng nhất về việc máy tính mô phỏng ^{Shutterstock} tư duy của con người. **Machine Learning** là một tập hợp con của AI, tập trung vào việc cho phép máy tính tự học hỏi từ dữ liệu thay vì được lập trình cứng nhắc. Cuối cùng, **Deep Learning** là một kỹ thuật tiên tiến của Machine Learning, sử dụng các mạng nơ-ron nhân tạo nhiều lớp để xử lý dữ liệu phức tạp. Sự phát triển từ AI truyền thống đến Deep Learning là một hành trình nâng cấp khả năng xử lý của máy móc, từ việc dựa vào các quy tắc do con người đặt ra đến khả năng tự nhận diện và phân tích dữ liệu ở mức độ tinh vi như não người.

2. Sự khác biệt giữa Machine Learning và Deep Learning

Dưới đây là 4-5 điểm khác biệt chính, tập trung vào khái niệm và yêu cầu dữ liệu:

- **Về bản chất khái niệm:** Machine Learning sử dụng các thuật toán thống kê và mô hình toán học (như hồi quy, cây quyết định) để tìm ra quy luật trong dữ liệu. Trong khi đó, Deep Learning là một cấp độ sâu hơn của ML, mô phỏng cấu trúc mạng nơ-ron của não người để giải quyết các vấn đề phức tạp hơn.
- **Về lượng dữ liệu yêu cầu:** Machine Learning có thể hoạt động tốt và đưa ra kết quả chính xác ngay cả với các tập dữ liệu có quy mô vừa và nhỏ. Ngược lại, Deep Learning đòi hỏi một lượng dữ liệu "khổng lồ" (Big Data) để huấn luyện; nếu dữ liệu quá ít, hiệu suất của Deep Learning sẽ bị sụt giảm nghiêm trọng.
- **Về cách xử lý đặc trưng (Feature Engineering):** Với Machine Learning, con người (các chuyên gia dữ liệu) phải can thiệp thủ công để xác định và trích xuất các đặc trưng quan trọng từ dữ liệu thô. Với Deep Learning, máy tính có khả năng tự động học các đặc trưng phức tạp từ dữ liệu thô mà không cần sự can thiệp của con người.
- **Về loại dữ liệu:** Machine Learning thường làm việc hiệu quả nhất với dữ liệu có cấu trúc (như bảng số liệu, Excel). Deep Learning lại vượt trội trong việc xử lý các loại dữ liệu phi cấu trúc như hình ảnh, âm thanh, video và văn bản tự nhiên.
- **Yêu cầu về hạ tầng (Phần cứng):** Do cấu trúc mạng nơ-ron nhiều lớp và lượng dữ liệu lớn, Deep Learning yêu cầu sức mạnh tính toán cực cao (thường cần GPU chuyên dụng), trong khi Machine Learning có thể chạy tốt trên các máy tính sử dụng CPU thông thường.

- **Prompt nâng cao (Role prompting + Output formatting):**

Đóng vai là một chuyên gia công nghệ đang viết bài trên blog hướng dẫn người mới. Hãy tóm tắt tài liệu chuyên sâu dưới đây về Machine Learning và Deep Learning.

Yêu cầu định dạng:

1. Bắt đầu bằng 1 câu định nghĩa khái quát, sử dụng ngôn ngữ đời thường, dễ hiểu.
2. Lập một bảng so sánh 3 điểm khác biệt cốt lõi nhất giữa hai khái niệm này.
3. Đưa ra một ví dụ thực tế (như nhận diện khuôn mặt hay dự đoán giá nhà) để minh họa.

Tài liệu: [Chèn tài liệu]

Đóng vai là một chuyên gia công nghệ đang viết bài trên blog hướng dẫn người mới. Hãy tóm tắt tài liệu chuyên sâu dưới đây về Machine Learning và Deep Learning....

Chào các bạn, mình là một chuyên gia công nghệ và hôm nay mình sẽ giúp các bạn "giải mã" hai khái niệm đang làm mưa làm gió trong giới kỹ thuật: **Machine Learning (Học máy)** và **Deep Learning (Học sâu)** một cách đơn giản nhất.

1. Định nghĩa khái quát

Nếu coi Trí tuệ nhân tạo (AI) là một cái cây lớn, thì **Machine Learning** chính là một cành cây khỏe mạnh giúp máy tính tự học từ kinh nghiệm thay vì phải cấm tay chỉ việc, còn **Deep Learning** là những chiếc lá ở ngọn cây – một kỹ thuật thông minh hơn giúp máy tính có thể tự bắt chước cách bộ não con người tự suy để xử lý những thông tin cực kỳ phức tạp.

2. Bảng so sánh 3 điểm khác biệt cốt lõi

Điểm khác biệt	Machine Learning (Học máy)	Deep Learning (Học sâu)
Sự can thiệp của con người	Cần con người chọn lọc và dán nhãn các đặc điểm dữ liệu quan trọng trước khi máy học (ví dụ: chỉ cho máy biết đâu là cái tai, đâu là cái mũi).	Máy tự động tìm ra các đặc điểm quan trọng mà không cần con người can thiệp sâu vào khâu xử lý ban đầu.
Quy mô dữ liệu	Hoạt động tốt với lượng dữ liệu vừa phải, có cấu trúc rõ ràng.	Cần một lượng dữ liệu khổng lồ (Big Data) để có thể đạt được độ chính xác cao.

Sức mạnh phần cứng

Có thể chạy trên các máy tính thông thường (CPU).

Đòi hỏi cấu hình máy tính cực mạnh, đặc biệt là các chip đồ họa (GPU) để xử lý hàng triệu phép tính cùng lúc.

Export to Sheets

NEURAL NETWORK

Input layer, Hidden layer, Output layer

Input connected nodes, Hidden connected nodes

Explore

3. Ví dụ minh họa thực tế: Nhận diện khuôn mặt

Hãy tưởng tượng bạn muốn dạy máy tính nhận diện khuôn mặt của mình để mở khóa điện thoại:

- Với Machine Learning:** Bạn sẽ phải "mớm" cho máy tính các thông số cụ thể: "Đây là khoảng cách giữa hai mắt", "Đây là độ cao của mũi", "Đây là hình dáng khuôn miệng". Máy tính sẽ dựa vào các chỉ số do bạn cung cấp để so sánh và xác nhận xem đó có phải là bạn hay không.
- Với Deep Learning:** Bạn chỉ cần cho máy "xem" hàng nghìn bức ảnh về khuôn mặt bạn ở các góc độ khác nhau. Hệ thống (gọi là mạng nơ-ron nhân tạo) sẽ tự phân tích các lớp dữ liệu: lớp đầu tiên nhận diện độ sáng tối, lớp tiếp theo nhận diện đường nét, lớp sau nữa nhận diện các bộ phận phức tạp... Cuối cùng, nó tự "hiểu" thế nào là khuôn mặt của bạn mà không cần bạn phải giải thích từng chi tiết nhỏ.

Hy vọng bài viết này giúp các bạn có cái nhìn rõ nét hơn về hai "người khổng lồ" trong giới công nghệ hiện nay. Đừng ngần ngại bắt đầu tìm hiểu vì đây chính là tương lai của chúng ta!

Tác vụ 2: Giải thích một khái niệm phức tạp

(Chủ đề: Khái niệm Lập trình Hướng đối tượng - OOP)

- Prompt cơ bản (Đơn giản, ngắn gọn):** Giải thích Lập trình Hướng đối tượng là gì.

Giải thích Lập trình Hướng đối tượng là gì.

Chào bạn! Rất vui được cùng bạn "mổ xẻ" khái niệm này. Đừng để cái tên nghe có vẻ đồ sộ của nó làm bạn lo lắng. Thực ra, **Lập trình hướng đối tượng (Object-Oriented Programming - OOP)** chỉ đơn giản là cách chúng ta đưa thế giới thực vào trong những dòng code thôi.

Thay vì viết một danh sách dài các bước cần làm (như một công thức nấu ăn), OOP tập trung vào các **"đối tượng" (Objects)** — những thực thể có đặc điểm và hành động riêng.

1. Hai khái niệm nền tảng: Lớp và Đối tượng

Để hiểu OOP, bạn cần phân biệt được **Lớp (Class)** và **Đối tượng (Object)**:

- Lớp (Class):** Là một "bản thiết kế" hoặc khuôn mẫu. Ví dụ: Bản vẽ kỹ thuật của một chiếc xe hơi. Nó quy định xe phải có màu sắc, thương hiệu và có thể chạy được.
- Đối tượng (Object):** Là một thực thể cụ thể được tạo ra từ khuôn mẫu đó. Ví dụ: Chiếc xe VinFast màu đỏ của bạn hay chiếc Toyota màu bạc của hàng xóm.

2. Bốn cột trụ chính của OOP

OOP giống như một ngôi nhà được giữ vững bởi 4 chiếc cột quan trọng sau đây:

1. Tính Đóng gói (Encapsulation)

Hãy tưởng tượng một viên thuốc. Bạn chỉ cần uống nó mà không cần biết các thành phần hóa học bên trong được trộn lẫn thế nào. Trong lập trình, tính đóng gói giúp **che giấu thông tin nội bộ** và chỉ đưa ra những gì cần thiết để tương tác. Điều này bảo vệ dữ liệu không bị thay đổi tùy tiện.

2. Tính Kế thừa (Inheritance)

Nếu bạn đã có lớp "Động vật", bạn không cần định nghĩa lại "biết thở" hay "biết ăn" cho lớp "Con Chó". Lớp "Con Chó" sẽ **kế thừa** toàn bộ đặc điểm từ lớp "Động vật" và chỉ cần thêm các đặc điểm riêng của nó như "biết sủa". Việc này giúp tái sử dụng mã nguồn cực kỳ hiệu quả.

3. Tính Đa hình (Polymorphism)

"Đa hình" có nghĩa là một hành động có thể được thực hiện theo nhiều cách khác nhau. Cùng là hành động "Kêu", nhưng Con Chó thì sủa "Gâu gâu", còn Con Mèo thì kêu "Meo meo". Trong code, bạn có thể gọi cùng một hàm nhưng mỗi đối tượng sẽ phản hồi theo cách riêng của nó.

4. Tính Trừu tượng (Abstraction)

Khi bạn lái xe, bạn chỉ cần biết đạp ga là xe chạy, đạp phanh là xe dừng. Bạn không cần biết piston hoạt động ra sao hay xăng được đốt cháy thế nào. Tính trừu tượng giúp chúng ta **tập trung vào những gì đối tượng làm thay vì cách nó thực hiện**.

3. Tại sao chúng ta cần dùng OOP?

Tại sao giới lập trình lại "cuồng" OOP đến vậy? Câu trả lời nằm ở 3 chữ: **Đễ quản lý**.

- Tái sử dụng:** Viết một lần, dùng nhiều nơi (nhờ kế thừa).
- Đễ bảo trì:** Khi có lỗi ở phần "Bánh xe", bạn chỉ cần sửa ở lớp Bánh xe, không làm ảnh hưởng đến cả "chiếc xe".
- Sát với thực tế:** Giúp lập trình viên tư duy logic giống như cách chúng ta nhìn nhận thế giới xung quanh.

Bạn đang dự định học ngôn ngữ lập trình nào để thực hành OOP (như Java, C++, hay Python)?
Mình có thể giúp bạn viết thử một ví dụ đơn giản bằng ngôn ngữ đó!

- **Prompt cải tiến (Chi tiết hơn, có cấu trúc):** Hãy giải thích chi tiết 4 tính chất cơ bản của Lập trình Hướng đối tượng (OOP) bao gồm: Đóng gói, Kế thừa, Đa hình và Trừu tượng. Lấy ví dụ minh họa cho mỗi tính chất.

Hãy giải thích chi tiết 4 tính chất cơ bản của Lập trình Hướng đối tượng
(OOP) bao gồm: Đóng gói, Kế thừa, Đa hình và Trừu tượng.
Lấy ví dụ minh họa cho mỗi tính chất.

Chào bạn! Chào mừng bạn đến với thế giới của **Lập trình Hướng đối tượng (OOP)**. Đây không chỉ là một phương pháp lập trình, mà còn là một cách tư duy giúp chúng ta mô hình hóa những thứ phức tạp trong thế giới thực vào trong code một cách gọn gàng và dễ quản lý hơn.

Dưới đây là chi tiết về "tứ trụ" của OOP mà bất kỳ lập trình viên nào cũng cần nắm lòng:

1. Tính Đóng gói (Encapsulation)

Định nghĩa: Là khả năng che giấu thông tin bên trong và chỉ lộ ra những gì cần thiết thông qua các phương thức giao tiếp công khai. Nó giúp bảo vệ trạng thái nội bộ của đối tượng khỏi những tác động sai lệch từ bên ngoài.

- Đặc điểm:** Thường sử dụng các từ khóa truy cập như `private`, `protected`, `public`. Các biến thường được để là `private` và truy cập qua các hàm `getter` và `setter`.
- Ví dụ:** Hãy tưởng tượng chiếc **Viên thuốc**. Bạn chỉ cần biết uống nó để chữa bệnh, còn các thành phần hóa học bên trong được bao bọc bởi lớp vỏ. Bạn không thể tự ý thay đổi tỉ lệ hóa chất bên trong viên thuốc nếu không có sự cho phép của nhà sản xuất.

2. Tính Kế thừa (Inheritance)

Định nghĩa: Cho phép một lớp (lớp con) thừa hưởng lại các đặc tính (thuộc tính và phương thức) từ một lớp khác (lớp cha) đã tồn tại.

- Đặc điểm:** Giúp tái sử dụng mã nguồn, tránh việc viết lại những đoạn code giống nhau. Nó tạo ra một mối quan hệ "là một" (is-a).
- Ví dụ:** Ta có lớp cha là **Động vật**. Các lớp con như **Chó**, **Mèo**, **Chim** đều kế thừa các đặc tính chung như `cần_nạp`, `màu_lông` và hành động `ăn()`, `ngủ()`. Tuy nhiên, mỗi loài có thể có thêm các đặc điểm riêng (Chim thì có `sải_cánh`).

3. Tính Đa hình (Polymorphism)

Định nghĩa: Cho phép các đối tượng khác nhau thực thi cùng một thống điệp (hành động) theo những cách khác nhau.

- Đặc điểm:** Có hai dạng chính là **Overloading** (Nạp chồng - cùng tên hàm nhưng khác tham số) và **Overriding** (Ghi đè - lớp con viết lại hàm của lớp cha).
- Ví dụ:** Lớp cha **Động vật** có phương thức `phát_ra_tiếng_kêu()`.
 - Đối tượng **Chó** khi thực hiện hàm này sẽ kêu "Gâu gâu".
 - Đối tượng **Mèo** khi thực hiện hàm này sẽ kêu "Meo meo".Cùng là một hành động "kêu", nhưng mỗi loài lại có cách thể hiện khác nhau.

4. Tính Trừu tượng (Abstraction)

Định nghĩa: Là quá trình chọn lọc ra các đặc điểm quan trọng nhất của đối tượng cần quản lý và bỏ qua những chi tiết rườm rà, không cần thiết.

- Đặc điểm:** Tập trung vào câu hỏi "Đối tượng làm gì?" thay vì "Nó làm như thế nào?". Trong code, điều này thường được thể hiện qua `Abstract Class` hoặc `Interface`.
- Ví dụ:** Khi bạn lái **Xe máy**, bạn chỉ cần quan tâm đến `tay ga`, `phanh`, và `nút đề`. Bạn không cần biết piston hoạt động ra sao hay xăng được phun vào buồng đốt như thế nào để xe có thể chạy. Những chi tiết kỹ thuật đó đã được "trừu tượng hóa" đi.

Bảng tổng kết nhanh		
Tính chất	Từ khóa chính	Mục đích chính
Đóng gói	Che giấu, Bảo mật	Bảo vệ dữ liệu, giảm sự phụ thuộc.
Kế thừa	Tái sử dụng	Giảm thiểu code trùng lặp, tạo hệ thống phân cấp.
Đa hình	Ghi đè, Nạp chồng	Linh hoạt trong việc xử lý các loại đối tượng khác nhau.
Trừu tượng	Đơn giản hóa	Giảm sự phức tạp, tập trung vào giao diện tương tác.

Export to Sheets

Hy vọng cách giải thích này giúp bạn hình dung rõ hơn về sức mạnh của OOP! Bạn đang định áp dụng những kiến thức này vào ngôn ngữ lập trình cụ thể nào (như Java, Python hay C++) không?

- **Prompt nâng cao (Chain-of-thought + Persona):**

Bạn là một giảng viên dạy lập trình siêu tâm lý và giỏi truyền đạt. Hãy giải thích khái niệm "Lập trình hướng đối tượng (OOP)" cho một sinh viên IT năm nhất đang cảm thấy rất bối rối.

Hãy suy nghĩ và trình bày theo các bước sau:

- Bước 1: Dùng một hình ảnh ẩn dụ trong đời sống (ví dụ: Bản thiết kế ngôi nhà và những ngôi nhà được xây lên) để giải thích khái niệm Class và Object.
- Bước 2: Giải thích ngắn gọn 4 tính chất của OOP dựa trên chính ẩn dụ ở Bước 1 để sinh viên dễ liên kết.
- Bước 3: Cung cấp một đoạn code Python ngắn gọn nhất có thể để minh họa việc tạo một Class đơn giản.

- **Prompt cải tiến (Chi tiết hơn, có cấu trúc):** Tôi đang ôn thi môn Python. Hãy tạo một bài kiểm tra ngắn gồm 5 câu hỏi trắc nghiệm về lệnh if/else và vòng lặp for/while. Cung cấp đáp án ở cuối.

Tôi đang ôn thi môn Python. Hãy tạo một bài kiểm tra ngắn gồm 5 câu hỏi trắc nghiệm về lệnh if/else và vòng lặp for/while. Cung cấp đáp án ở cuối.

Chúc bạn ôn thi thật tốt nhé! Python là một ngôn ngữ rất thú vị, và việc nắm vững cấu trúc điều khiển (if/else) cùng vòng lặp là bạn đã nắm được "xương sống" của tư duy lập tức rồi đấy.

Dưới đây là 5 câu hỏi trắc nghiệm được thiết kế để kiểm tra các lỗi thường gặp:

Bài Kiểm Tra Ngắn: Python Control Flow

Câu 1: Kết quả của đoạn mã sau là gì?

```
Python
x = 10
if x > 5:
    print("A")
elif x > 8:
    print("B")
else:
    print("C")
```

- A. A
- B. B
- C. A và B
- D. C

Câu 5: Kết quả của đoạn mã sau là gì?

```
Python
for i in range(3):
    for j in range(2):
        if i == j:
            print(i, j)
```

- A. 0 0 và 11
- B. 0 0, 11, và 22
- C. 0 0
- D. Không in ra gì cả

Đáp án & Giải thích chi tiết

Câu	Đáp án	Giải thích nhanh
-----	--------	------------------

Câu 2: Trong vòng lặp `for i in range(1, 10, 2):`, giá trị cuối cùng của `i` được in ra là bao nhiêu?

- A. 10
- B. 9
- C. 8
- D. 11

Câu 3: Đoạn mã sau đây sẽ in ra màn hình bao nhiêu lần chữ "Hello"?

```
Python
count = 0
while count < 3:
    print("Hello")
    count += 1
```

- A. 2
- B. 3
- C. 4
- D. Lặp vô hạn

Câu 4: Lệnh nào dùng để dừng hoàn toàn vòng lặp ngay lập tức dù điều kiện vẫn đang đúng?

- A. `continue`
- B. `pass`
- C. `break`

Đáp án & Giải thích chi tiết

Câu	Đáp án	Giải thích nhanh
1	A	Trong cấu trúc <code>if-elif-else</code> , Python sẽ dừng lại ngay khi tìm thấy điều kiện đúng đầu tiên. Dù <code>x > 8</code> cũng đúng, nhưng khối <code>if x > 5</code> đã được thực thi trước.
2	B	<code>range(start, stop, step)</code> chạy từ <code>start</code> đến <code>stop - 1</code> . Với bước nhảy là 2, dãy số là 1, 3, 5, 7, 9.
3	B	Vòng lặp chạy khi <code>count</code> bằng 0, 1, 2. Khi <code>count</code> đạt bằng 3, điều kiện <code>3 < 3</code> sai nên dừng lại.
4	C	<code>break</code> thoát khỏi vòng lặp; <code>continue</code> chỉ bỏ qua các lệnh còn lại của lần lặp hiện tại để sang lần lặp mới.
5	A	Vòng lặp lồng nhau: Khi <code>i=0</code> , <code>j</code> chạy 0, 1 (khớp 0 0). Khi <code>i=1</code> , <code>j</code> chạy 0, 1 (khớp 1 1). Khi <code>i=2</code> , <code>j</code> chạy 0, 1 (không có cặp nào bằng nhau).

- **Prompt nâng cao (Context + Ràng buộc mức độ + Formatting):**

Tôi chuẩn bị thi giữa kỳ môn Nhập môn Lập trình Python. Đóng vai người ra đề, hãy tạo một bộ câu hỏi ôn tập chuyên sâu về "Cấu trúc điều kiện và Vòng lặp".

Cấu trúc bộ câu hỏi phải bao gồm:

- Phần 1: 3 câu hỏi trắc nghiệm kiểm tra lý thuyết (Mức độ Dễ).
- Phần 2: 2 đoạn code Python bị lỗi logic hoặc cú pháp. Yêu cầu sinh viên tìm và giải thích lỗi (Mức độ Trung bình).
- Phần 3: 1 bài toán thực tế yêu cầu tự viết code giải quyết (Mức độ Khó).

Lưu ý: Hãy trình bày rõ ràng, in đậm các từ khóa. Phần đáp án và giải thích

chi tiết vui lòng đặt ở cuối cùng, tách biệt với phần câu hỏi.

Tôi chuẩn bị thi giữa kỳ môn Nhập môn Lập trình Python. Đóng vai người ra đề, hãy tạo một bộ câu hỏi ôn tập chuyên sâu về "Cấu trúc điều kiện và Vòng lặp"....

Chào bạn, với tư cách là người ra đề, tôi đã biên soạn bộ câu hỏi ôn tập chuyên sâu về **Cấu trúc điều kiện (if-elif-else)** và **Vòng lặp (for , while)** trong Python. Bộ đề này được thiết kế theo đúng cấu trúc và độ khó mà bạn yêu cầu để chuẩn bị tốt nhất cho kỳ thi giữa kỳ.

BỘ CÂU HỎI ÔN TẬP NHẬP MÔN LẬP TRÌNH PYTHON

Chủ đề: Cấu trúc điều kiện và Vòng lặp

PHẦN 1: TRẮC NGHIỆM LÝ THUYẾT (Mức độ: Dễ)

Câu 1: Trong Python, cấu trúc nào sau đây được dùng để kiểm tra nhiều điều kiện rẽ nhánh thay thế cho cấu trúc `switch-case` ở các ngôn ngữ khác?

A. `else if`
B. `elif`
C. `ifelse`
D. `when`

Câu 2: Kết quả của đoạn mã sau là gì?

```
Python
x = 10
if x > 5:
```

```
Python
x = 10
if x > 5:
    print("A", end="")
if x > 8:
    print("B", end="")
else:
    print("C", end="")
```

A. AB
B. A
C. B
D. AC

Câu 3: Lệnh nào dùng để kết thúc ngay lập tức vòng lặp hiện tại và chuyển đến câu lệnh kế tiếp bên ngoài vòng lặp?

A. `continue`
B. `pass`
C. `break`
D. `return`

PHẦN 2: PHÂN TÍCH VÀ SỬA LỖI CODE (Mức độ: Trung bình)

Đoạn code 1: Lỗi cú pháp/Logic

Yêu cầu: Tìm lỗi khiến chương trình không chạy được hoặc chạy sai mục đích (tính tổng các số chẵn từ 1 đến 10).

```
Python
tong = 0
for i in range(1, 10):
```

```
Python
tong = 0
for i in range(1, 10):
    tong += i
print("Tổng các số chẵn là:", tong)
```

Đoạn code 2: Lỗi Logic (Vòng lặp và kiện)

Yêu cầu: Giải thích tại sao đoạn code sau không bao giờ dừng lại và cách khắc phục.

```
Python
n = 5
while n > 0:
    print("Giá trị của n là:", n)
    if n == 1:
        print("Bỏ xong rồi!")
```

PHẦN 3: BÀI TOÁN THỰC TẾ (Mức độ: Khó)

Đề bài: Viết một chương trình quản lý hệ thống giảm giá cho một cửa hàng.

Yêu cầu: Giải thích tại sao đoạn code sau không bao giờ dừng lại và cách khắc phục.

- Nhập vào tổng số tiền hóa đơn (số nguyên).
- Nhập vào loại thẻ khách hàng (Thân thiết, VIP, hoặc Khách lạ).
- Áp dụng quy tắc:
 - Nếu là khách VIP: Giảm 20% tổng hóa đơn.
 - Nếu là khách Thân thiết: Giảm 10% nếu hóa đơn trên 500.000đ, ngược lại giảm 5%.
 - Nếu là khách lạ: Không giảm giá trừ khi hóa đơn trên 1000.000đ thì giảm 2%.

4. Sử dụng vòng lặp để chương trình cho phép nhập liên tục nhiều hóa đơn cho đến khi người dùng nhập "thoát".

ĐÁP ÁN VÀ GIẢI THÍCH CHI TIẾT

Đáp Án Phần 1:

- B. `elif`: Trong Python, từ khóa rẽ nhánh của "nếu IP" là `elif`.
- A. AB: Vì cả hai điều kiện `x > 5` và `x > 8` đều đúng (`True`), và đây là hai khối `if` độc lập, khối `else` chỉ gắn với `if x > 8`.
- C. `break`: Lệnh `break` thoát hoàn toàn vòng lặp, còn `continue` chỉ bỏ qua các lệnh còn lại trong lần lặp hiện tại để sang lần lặp mới.

Đáp Án Phần 2:

- Đoạn code 1 - Lỗi: Sử dụng một dấu bằng `=` thay vì hai dấu bằng `==` (phép so sánh).
 - Lỗi khóa: `range(1, 10)` chỉ chạy đến 9, nếu muốn tính cả số 10 phải dùng `range(1, 11)`.
- Đoạn code 2 - Lỗi: Thiếu bước cập nhật biến điều kiện. Biến `n` luôn bằng 5, điều kiện `n > 0` luôn đúng dẫn đến vòng lặp vô tận.
 - Khắc phục: Thêm `n -= 1` (giảm n sau mỗi lần lặp) vào bên trong khối `while`.

Đáp Án Phần 3 (Gợi ý code):

```
Python
while True:
    input_str = input("Nhập tổng hóa đơn (hoặc gõ 'thoát' để dừng): ")
    if input_str.lower() == 'thoát':
        break

    tong_tien = int(input_str)
    loai_the = input("Nhập loại thẻ (VIP/Thân thiết/Lạ): ").strip().lower()
    giam_gia = 0

    if loai_the == "vip":
        giam_gia = 0.2
    elif loai_the == "than thiet":
        giam_gia = 0.1 if tong_tien > 500000 else 0.05
    elif loai_the == "la":
        if tong_tien > 1000000:
            giam_gia = 0.02

    so_tien_cuai = tong_tien * (1 - giam_gia)
    print(f"Số tiền sau giảm giá: {so_tien_cuai:,.0f} VNĐ")
    print("-" * 20)
```

Chúc bạn ôn tập thật tốt và đạt kết quả cao trong kỳ thi sắp tới!

III. Thử nghiệm và So sánh Kết quả (Sử dụng ChatGPT/Gemini)

Tác vụ	Loại Prompt	Đánh giá chất lượng & Kết quả tóm tắt
1. Tóm tắt tài liệu	Cơ bản	AI chỉ rút gọn chữ, văn bản vẫn khô khan, khó nắm bắt được trọng tâm khác biệt.
	Cải tiến	Kết quả mạch lạc hơn với

		gạch đầu dòng, tuy nhiên vẫn mang tính hàn lâm, học thuật.
	Nâng cao	Rất tốt. Bảng so sánh trực quan, sinh động. Ví dụ thực tế giúp người không chuyên cũng hiểu được ngay lập tức.
2. Giải thích khái niệm	Cơ bản	Trả về định nghĩa giống hết sách giáo khoa hoặc Wikipedia. Gây nhàm chán và khó hiểu.
	Cải tiến	Giải thích đầy đủ 4 tính chất, có cấu trúc tốt, nhưng thiếu sự liên kết giữa các khái niệm.
	Nâng cao	Xuất sắc. Ấn dụ "Bản thiết kế ngôi nhà" giúp hình dung dễ dàng. AI tuân thủ đúng trình tự tư duy (Chain-of-thought) từ đời thực đến code.
3. Tạo câu hỏi ôn tập	Cơ bản	Đưa ra vài bài toán ngẫu nhiên, không có hệ thống kiểm tra kiến thức.
	Cải tiến	Đủ số lượng câu hỏi trắc nghiệm để luyện tập nhanh, đáp ứng nhu cầu cơ bản.
	Nâng cao	Xuất sắc. Đề cương ôn tập toàn diện (trắc nghiệm, tìm lỗi, viết code). Phần tìm lỗi code (debug) cực kỳ hữu ích cho tư duy lập trình viên.

IV. Phân tích lý do tại sao một số prompt hiệu quả hơn

Từ các trải nghiệm trên, có thể thấy rõ các "Prompt nâng cao" mang lại kết quả vượt trội vì:

1. **Chỉ định vai trò (Role-playing/Persona):** Việc yêu cầu AI đóng vai "Giảng viên tâm lý" hay "Chuyên gia viết blog" giúp thay đổi hoàn toàn văn phong. AI biết cách sử dụng ngôn từ gần gũi, phù hợp với người học thay vì giọng điệu "robot" mặc định.
2. **Định hình cấu trúc đầu ra (Output formatting):** Ràng buộc AI phải trả lời bằng bảng biểu, danh sách, hay chia phần rõ ràng (trong tác vụ 3) giúp giảm thời gian người dùng phải tự định dạng lại thông tin.
3. **Hướng dẫn tư duy từng bước (Chain-of-thought):** Thay vì bắt AI đưa ra kết quả ngay, việc ép AI đi theo trình tự (Ví dụ thực tế -> Khái niệm -> Code minh họa) giúp logic của câu trả lời chặt chẽ và thuyết phục hơn rất nhiều.
4. **Cung cấp bối cảnh (Context):** Nói cho AI biết mục đích (ví dụ: "Tôi đang ôn thi giữa kỳ") giúp AI tự động điều chỉnh độ khó và nội dung trọng tâm cho phù hợp.

V. Tổng hợp nguyên tắc và mẹo viết prompt hiệu quả

Dựa trên kết quả thử nghiệm, em rút ra các nguyên tắc cốt lõi khi giao tiếp với LLMs:

- **Rõ ràng hơn là ngắn gọn:** Đừng tiếc chữ. Một prompt dài, mô tả chi tiết yêu cầu luôn mang lại kết quả tốt hơn một câu lệnh cộc lốc.
- **Công thức "Vai trò + Bối cảnh + Nhiệm vụ + Định dạng":** Đây là bộ khung chuẩn để xây dựng mọi prompt phức tạp.
 - *Ai?* (Đóng vai chuyên gia).
 - *Trong hoàn cảnh nào?* (Cần ôn thi).
 - *Làm gì?* (Tạo đề cương).
 - *Như thế nào?* (Dạng trắc nghiệm, in đậm từ khóa).
- **Sử dụng dấu phân cách:** Khi yêu cầu AI xử lý một đoạn văn bản (như dịch thuật, tóm tắt), hãy dùng dấu ngoặc kép "" hoặc ngoặc vuông [] để tách biệt phần lệnh của người dùng và phần văn bản dữ liệu, giúp AI không bị nhầm lẫn.
- **Tinh chỉnh liên tục (Iterative):** Nếu kết quả lần 1 chưa ưng ý, hãy phản hồi lại cho AI (ví dụ: "Hãy làm cho nó ngắn hơn nữa", "Ví dụ này hơi khó, đổi ví dụ khác").